

Git & GitHub সম্পূর্ণ মাস্টার গাইড

প্রো-লেভেল কমান্ডস, ওয়ার্কিং মেকানিজম এবং প্রম্পট এক্সাম্পল ম্যানুয়াল

এই ডকুমেন্টে গিটের বেসিক থেকে শুরু করে অ্যাডভান্সড প্লাম্বিং (Plumbing) এবং পোরসেলিন (Porcelain) কমান্ডসহ সবকিছুর বিস্তারিত মেকানিজম এবং প্রম্পট উদাহরণ দেওয়া হলো।

১. সেটআপ এবং কনফিগারেশন (Setup & Configuration)

কমান্ড (Command)	কার্যপ্রণালী (Working Mechanism)	প্রম্পট উদাহরণ (Prompt Example)
<code>git config</code>	গিটের গ্লোবাল বা লোকাল সেটিংস (যেমন ইউজারনেম, ইমেইল, ডিফল্ট এডিটর) কনফিগার করে।	<pre>\$ git config --global user.name "Sourav Golder" \$ git config --global user.email "sourav@example.com"</pre>
<code>git init</code>	বর্তমান ফোল্ডারে একটি সম্পূর্ণ নতুন এবং খালি লোকাল গিট রিপোজিটরি (<code>.git</code> হিডেন ডিরেক্টরি) তৈরি করে।	<pre>\$ mkdir my-project \$ cd my-project \$ git init</pre>
<code>git clone</code>	রিমোট সার্ভার (যেমন GitHub) থেকে একটি প্রজেক্টের সম্পূর্ণ হিস্ট্রি ও ফাইল লোকাল মেশিনে কপি করে নামায়।	<pre>\$ git clone https://github.com/user/repo.git</pre>

২. ফাইল ট্র্যাকিং এবং স্ন্যাপশট (Basic Workflow & Snapshotting)

কমান্ড (Command)	কার্যপ্রণালী (Working Mechanism)	প্রম্পট উদাহরণ (Prompt Example)
<code>git status</code>	ওয়ার্কিং ডিরেক্টরি এবং স্টেজিং এরিয়ার বর্তমান অবস্থা দেখায় (কোন ফাইলগুলো পরিবর্তিত, আনট্র্যাকড বা স্টেজড)।	<pre>\$ git status</pre>
<code>git add</code>	ফাইলের পরিবর্তনগুলোকে ওয়ার্কিং ডিরেক্টরি থেকে Staging Area-তে যোগ করে (কমিটের জন্য প্রস্তুত করে)।	<pre>\$ git add index.html \$ git add .</pre>
<code>git commit</code>	Staging Area-তে থাকা সমস্ত ফাইলকে একটি ইউনিক আইডি (SHA-1 হ্যাশ) সহ স্থায়ীভাবে লোকাল রিপোতে সেভ করে।	<pre>\$ git commit -m "feat: add login authentication"</pre>
<code>git rm</code>	ফাইলকে ওয়ার্কিং ডিরেক্টরি এবং গিট ট্র্যাকিং উভয় জায়গা থেকে মুছে ফেলে।	<pre>\$ git rm old_file.txt</pre>
<code>git mv</code>	কোনো ফাইলের নাম পরিবর্তন বা ফাইলকে অন্য ফোল্ডারে মুভ করে এবং গিটকে সেই পরিবর্তন ট্র্যাক করতে বলে।	<pre>\$ git mv config.json src/config.json</pre>

৩. ব্রাঞ্চিং, মার্জিং এবং অ্যাডভান্সড ইন্টিগ্রেশন (Branching, Merging & Advanced Integration)

কমান্ড (Command)	কার্যপ্রণালী (Working Mechanism)	প্রম্পট উদাহরণ (Prompt Example)
<code>git branch</code>	রেপোজিটরির সমস্ত ব্রাঞ্চের তালিকা দেখায় বা নতুন ব্রাঞ্চ তৈরি/মুছে ফেলে।	<pre>\$ git branch feature-ui \$ git branch -d feature-ui</pre>
<code>git checkout</code>	এক ব্রাঞ্চ থেকে অন্য ব্রাঞ্চে সুইচ করে অথবা কোনো নির্দিষ্ট কমিটের ফাইলে ফিরে যায়।	<pre>\$ git checkout feature-ui \$ git checkout -b feature-api</pre>
<code>git switch</code>	ব্রাঞ্চ পরিবর্তনের জন্য সুনির্দিষ্ট আধুনিক কমান্ড (checkout এর বিকল্প ও নিরাপদ সংস্করণ)।	<pre>\$ git switch main</pre>
<code>git merge</code>	নির্দিষ্ট কোনো ব্রাঞ্চের পুরো হিস্ট্রি ও কোড বর্তমান অ্যাক্টিভ ব্রাঞ্চের সাথে এনে মার্জ বা একত্রিত করে।	<pre>\$ git merge feature-ui</pre>
<code>git rebase</code>	বর্তমান ব্রাঞ্চের কমিটগুলোকে সাময়িকভাবে সরিয়ে মেইন ব্রাঞ্চের লেটেস্ট কমিটের মাথার ওপর নতুন করে সাজায় (লিনিয়ার হিস্ট্রির জন্য)।	<pre>\$ git rebase main</pre>
<code>git cherry-pick</code>	অন্য কোনো ব্রাঞ্চের সম্পূর্ণ কোড মার্জ না করে, শুধুমাত্র একটি সুনির্দিষ্ট একক কমিটকে (কমিট আইডি দিয়ে) টেনে এনে বর্তমান ব্রাঞ্চে অ্যাপ্লাই করে।	<pre>\$ git cherry-pick a1b2c3d4</pre>
<code>git tag</code>	ইতিহাসের কোনো সুনির্দিষ্ট গুরুত্বপূর্ণ কমিটকে একটি নাম বা ভার্সন লেবেল (যেমন রিলিজ ভার্সন) দিয়ে মার্ক করে রাখে।	<pre>\$ git tag -a v1.0.0 -m "Production Release v1.0"</pre>

৪. রিমোট ইন্টারঅ্যাকশন ও শেয়ারিং (Remote Sharing & Collaboration)

কমান্ড (Command)	কার্যপ্রণালী (Working Mechanism)	প্রম্পট উদাহরণ (Prompt Example)
<code>git remote</code>	লোকাল প্রজেক্টের সাথে সংযুক্ত দূরবর্তী সার্ভার বা গিটহাব লিংকের সংযোগ তৈরি, পরিবর্তন বা ভিউ করে।	<pre>\$ git remote add origin https://github.com/user/ repo.git \$ git remote -v</pre>
<code>git push</code>	লোকাল রেপোজিটরিতে সেভ করা সমস্ত লেটেস্ট কমিট গিটহাবের রিমোট সার্ভারে আপলোড করে।	<pre>\$ git push -u origin main</pre>
<code>git fetch</code>	রিমোট সার্ভার থেকে লেটেস্ট সমস্ত পরিবর্তনের মেটাডাটা ও হিস্ট্রি লোকাল গিটে ডাউনলোড করে, তবে কারেন্ট ফাইলে মার্জ করে না।	<pre>\$ git fetch origin</pre>
<code>git pull</code>	রিমোট সার্ভার থেকে লেটেস্ট কোড সরাসরি ডাউনলোড করে একই সাথে লোকাল ফাইলে মার্জ করে আপডেট করে দেয়।	<pre>\$ git pull origin main</pre>

কমান্ড (Command)	কার্যপ্রণালী (Working Mechanism)	প্রম্পট উদাহরণ (Prompt Example)
<code>git submodule</code>	একটি মেইন গিট প্রজেক্টের ভেতরে অন্য আরেকটি স্বাধীন গিট প্রজেক্ট বা রেপোজিটরি ট্র্যাকিং সহ যুক্ত করে।	<pre>\$ git submodule add https://github.com/other/lib.git</pre>

৫. ইতিহাস পর্যবেক্ষণ ও কোড অডিট (Inspection & Code Auditing)

কমান্ড (Command)	কার্যপ্রণালী (Working Mechanism)	প্রম্পট উদাহরণ (Prompt Example)
<code>git log</code>	প্রজেক্টের এ পর্যন্ত হওয়া সমস্ত কমিটের সম্পূর্ণ ক্রমানুসারিক ইতিহাস বা লগ প্রদর্শন করে।	<pre>\$ git log --oneline --graph --all</pre>
<code>git diff</code>	দুটি ফাইল, দুটি কমিট বা ওয়ার্কিং ডিরেক্টরির সাথে স্টেজিং এরিয়ার লাইন-বাই-লাইন কোডের পার্থক্য দেখায়।	<pre>\$ git diff \$ git diff main..feature-ui</pre>
<code>git show</code>	নির্দিষ্ট কোনো একটি কমিট বা অবজেক্টের বিস্তারিত তথ্য এবং সেই কমিটে করা কোডের নিখুঁত পরিবর্তন দেখায়।	<pre>\$ git show a1b2c3d4</pre>
<code>git blame</code>	একটি নির্দিষ্ট ফাইলের প্রতিটি লাইন কে, কখন এবং কোন কমিটের মাধ্যমে মডিফাই করেছে তা ট্র্যাক করে দেখায়।	<pre>\$ git blame src/auth.js</pre>
<code>git bisect</code>	বাইনারি সার্চ অ্যালগরিদম ব্যবহার করে কোডের কোন কমিটের কারণে প্রথম বাগ বা এরর এসেছে তা স্বয়ংক্রিয়ভাবে খুঁজে বের করে।	<pre>\$ git bisect start \$ git bisect bad \$ git bisect good v1.0.0</pre>

৬. ভুল সংশোধন, স্ট্যাশিং ও রিকভারি (Undoing, Stashing & Recovery)

কমান্ড (Command)	কার্যপ্রণালী (Working Mechanism)	প্রম্পট উদাহরণ (Prompt Example)
<code>git restore</code>	ওয়ার্কিং ডিরেক্টরির কোনো ফাইলে করা সাম্প্রতিক লোকাল কোড পরিবর্তন বাতিল করে শেষ কমিটের অবস্থায় নিয়ে যায়।	<pre>\$ git restore app.js</pre>
<code>git reset</code>	কারেন্ট হেডকে (HEAD) নির্দিষ্ট কোনো পূর্বের কমিটে নিয়ে যায়। --hard দিলে ফাইল চেঞ্জ মুছে যায়, --soft দিলে কোড থেকে যায়।	<pre>\$ git reset --soft HEAD~1 \$ git reset --hard a1b2c3d4</pre>
<code>git revert</code>	কোনো পূর্বের ভুল কমিটের পরিবর্তনকে উল্টে দিয়ে সম্পূর্ণ নতুন একটি সেফ কমিট তৈরি করে, যাতে আগের হিস্ট্রি ডিলিট না হয়।	<pre>\$ git revert e5f6g7h8</pre>
<code>git stash</code>	বর্তমান অসম্পূর্ণ বা আনকমিটেড কাজগুলোকে সাময়িকভাবে একটি ব্যাকআপ স্লটে লুকিয়ে রাখে যেন ক্লিন ডিরেক্টরিতে অন্য কাজ করা যায়।	<pre>\$ git stash \$ git stash pop</pre>

কমান্ড (Command)	কার্যপ্রণালী (Working Mechanism)	প্রম্পট উদাহরণ (Prompt Example)
<code>git clean</code>	ওয়ার্কিং ডিরেক্টরি থেকে সমস্ত আনট্র্যাকড (Untracked) অর্থাৎ গিটে যোগ না করা ফালতু বা অতিরিক্ত ফাইল মুছে ফেলে।	<pre>\$ git clean -fd</pre>
<code>git reflog</code>	লোকাল রেপোতে হেডের (HEAD) প্রতিটি মুভমেন্ট ও পরিবর্তনের নিখুঁত লগ রাখে। এর মাধ্যমে ডিলিট হওয়া কমিট বা ব্রাঞ্চও রিকভার করা সম্ভব।	<pre>\$ git reflog</pre>

৭. রেপো রক্ষণাবেক্ষণ (Repository Maintenance)

কমান্ড (Command)	কার্যপ্রণালী (Working Mechanism)	প্রম্পট উদাহরণ (Prompt Example)
<code>git gc</code>	Garbage Collector; অপ্রয়োজনীয় বা লুজ ফাইল অবজেক্ট মুছে রেপোজিটরিকে অপ্টিমাইজ এবং সাইজ ছোট করে।	<pre>\$ git gc --prune=now</pre>
<code>git fsck</code>	File System Check; ডাটাবেজের অবজেক্টগুলোর কানেক্টিভিটি এবং ইন্টিগ্রিটি বা বৈধতা চেক করে কোনো ত্রুটি আছে কিনা দেখে।	<pre>\$ git fsck --full</pre>